

ANOGAN i wykrywanie fake newsów z wykorzystaniem LLM, oraz czy to ma sens ?

Oskar Bartoszyński

7 czerwca 2025

Spis treści

1	Przedmowa	3
2	Wstęp	3
2.1	Architektura ANOGAN i proces trenowania/testowania	3
2.2	ANOGAN w detekcji anomalii i kontekst tekstowy	4
2.3	Badania i projekty: ANOGAN oraz warianty w wykrywaniu dezinformacji	4
2.4	Połączenia ANOGAN z modelami LLM w wykrywaniu fake newsów	5
2.5	Ograniczenia i kierunki przyszłych badań	5
3	implementacja modelu ANOGAN z LSTM	6
3.1	Implementacja LSTM i próba stworzenia mechanizmu Attention w NumPy	6
3.2	Tokenizacja i uproszczony BPE	7
3.3	Architektury Generatora i Dyskryminatora w PyTorch	7
3.4	Wczytanie i podział danych	8
3.5	Zastosowanie tokenizacji i BPE	8
3.6	Warstwa embeddingów	8
3.7	Ekstrakcja stanów ukrytych z LSTM+Attention	9
3.8	Scalanie i normalizacja	9
3.9	Przygotowanie DataLoaderów	9
3.10	Inicjalizacja modeli, optymalizatorów i funkcji straty	9
3.11	Funkcja <code>train_step</code>	10
4	Pętla treningowa	10
4.1	Detekcja anomalii i wizualizacja rozkładu	10
5	Problemy, spostrzeżenia, dokładniejsze opisy	11
6	Istnieje lepszy sposób ?	17
6.1	Ulepszanie modelu	17
6.2	Zmiany jakie nastąpiły	17

1 Przedmowa

Pomysł na ten projekt zrodził się podczas rozmowy z dr Pawlickim na temat sieci neuronowych. Rozmawialiśmy o tym, jak komputer uczy się rozpoznawać obiekty za pomocą sieci neuronowych, a on uświadomił mi, że komputer „zaakceptuje” wszystko, o ile będzie miało logiczną reprezentację numeryczną.

W przypadku obrazów składają się one z pikseli, a piksele są po prostu liczbami. Komputer nie przejmuję się tym, co reprezentują liczby; zawsze będzie próbował znaleźć w nich wzorce. To doprowadziło mnie do pytania: jeśli sieci oparte na GAN, w tym ANOGAN, są zaprojektowane przede wszystkim do analizy obrazów, dlaczego nie można ich również używać do analizy sekwencji zdań i porównywania ich ze sobą?

Ten eksperyment ma na celu zbadanie, czy możliwe jest dostosowanie sieci ANOGAN do analizy tekstu, w szczególności do wykrywania fałszywych wiadomości. Hipoteza jest taka, że fałszywe wiadomości można traktować jako anomalie w przestrzeni prawdziwych artykułów informacyjnych, tak jak ANOGAN wykrywa anomalie w obrazach. Jest również możliwość zrobienia tego w odwrotny sposób czyli prawdziwe wiadomości są odbierane jako anomalie, wszystko zależy od podejścia i danych jakimi "karmimy" model, jednak efekt pozostanie ten sam. Aby to udowodnić:

1. Konwertuję dane tekstowe na reprezentacje numeryczne przy użyciu kodowania par bajtów (BPE) i osadzeń.
2. Trenuję ANOGAN na prawdziwych wiadomościach, aby nauczył się generować autentyczne wzorce tekstowe.
3. Używam błędu rekonstrukcji jako metryki wykrywania anomalii, sprawdzając, czy fałszywe artykuły informacyjne znacząco odbiegają od prawdziwych.

Celem jest ustalenie, czy niesupervised podejście, takie jak ANOGAN, pierwotnie zaprojektowane dla obrazów, może skutecznie wykrywać fałszywe wiadomości na podstawie ich struktury tekstowej. Jeśli się powiedzie, może to otworzyć drzwi do wykorzystania generatywnych modeli wykrywania anomalii w przetwarzaniu języka naturalnego (NLP) i innych zadaniach opartych na tekście.

2 Wstęp

2.1 Architektura ANOGAN i proces trenowania/testowania

Model ANOGAN składa się z dwóch konkurujących sieci: generatora i dyskriminatora, zwykle z głębokimi warstwami konwolucyjnymi (architektura DCGAN). Generator uczy się tworzyć przykłady „normalne”(np. realistyczne obrazy) z wektora latentnego, a dyskriminator ocenia, czy próbka jest rzeczywista czy wygenerowana. W fazie trenowania obie sieci grają w przeciwny proces, gdzie dyskriminator maksymalizuje poprawną klasyfikację danych rzeczywistych vs.

sztucznych, a generator uczy się oszukać dyskryminator poprzez minimalizację jego straty. Po zakończeniu treningu generator nauczył się rozkładu danych normalnych.

W fazie testowej dla nowego wejścia x ANOGAN poszukuje wektora z w przestrzeni latentnej, którego generowane wyjście $G(z)$ będzie jak najbardziej zbliżone do x . W tym celu definiuje się funkcję straty z dwóch składników: błąd rekonstrukcji („residual loss” $L_R = \|x - G(z)\|$) oraz miarę odchylenia cech w reprezentacji dyskryminatora („discrimination loss” $L_D = \|f(x) - f(G(z))\|$). Parametry z są dostosowywane iteracyjnie przez propagację wsteczną, aby minimalizować łączną stratę $L = (1 - \lambda)L_R + \lambda L_D$. Uzyskany w ten sposób „anomaly score” jest miarą różnicy x od rozkładu normalnego – duża wartość (nieudana rekonstrukcja) oznacza anomalię.

2.2 ANOGAN w detekcji anomalii i kontekst tekstowy

Wykrywanie anomalii polega na identyfikacji danych znacząco odbiegających od „normalnej” dystrybucji. W kontekście tekstowym anomalie mogą obejmować treści o złośliwym charakterze – np. mowa nienawiści, cyberprzemoc czy dezinformacja. ANOGAN w oryginale działał na obrazach medycznych, lecz koncepcja przenoszenia jej na dane tekstowe wymaga przetłumaczenia dyskretnego tekstu na przestrzeń ciągłą. Na przykład, jako metrykę odległości między tekstami wykorzystuje się odległość między ich osadzonymi wektorami (embeddingami). Jednocześnie sieciowe generowanie sekwencji słów wymaga specjalnych technik: proponowano łączenie wyrazów w „paczki tekstowe” umożliwiające propagację gradientu i działanie dyskryminatora.

Definicja anomalii w sekwencjach tekstowych często sprowadza się do tego, że przykład jest „niestandardowy” w odniesieniu do zbioru danych treningowych. Na przykład Nyarko et al. definiują anomalię jako punkt danych znacznie odbiegający od normalnego rozkładu, zakładając, że tylko „pozytywne” (normalne) przykłady są znane, a anomalie są rzadkie i nieznane. W praktyce więc podejścia oparte na ANOGAN dla tekstu starają się rekonstruować typowe zdania i mierzyć rozbieżność nowych wiadomości od tego modelu językowego.

2.3 Badania i projekty: ANOGAN oraz warianty w wykrywaniu dezinformacji

Bezpośrednie zastosowanie ANOGAN do wykrywania fake newsów jest w literaturze rzadko opisywane, ale pojawiły się próby użycia GAN-ów do wykrywania nieprawdziwych treści. Na przykład ARAE-ANOGAN (2020) to metoda tekstowej detekcji anomalii, która wykorzystuje adversarialnie regulowany auto-encoder (ARAE) do rekonstrukcji normalnych zdań i definiuje wynik anomalii na podstawie błędów rekonstrukcji i cech enkodera. Autor pracy pokazuje, że ARAE-ANOGAN efektywnie wykrywa nietypowe zdania jako anomalie.

Inny przykład, choć w dziedzinie recenzji (niekoniecznie fake news), to model FakeGAN (2018) zaproponowany do wykrywania fałszywych opinii. Używa on dwóch dyskryminatorów i generatora trenowanego w trybie wzmocniania (RL),

aby odróżnić recenzje szczere od fałszywych. Choć FakeGAN nie jest metodą typu ANOGAN, ilustruje próbę wykorzystania struktury GAN-ów w zadaniu klasyfikacji tekstu z fałszywymi informacjami. Istotnie, wyniki FakeGAN są porównywalne z najlepszymi klasyfikatorami typu supervised na zbiorze recenzji podróbek.

Wśród wariantów ANOGAN na obrazach warto wspomnieć o f-ANOGAN (2019), który przyspiesza fazę testową poprzez wytrenowanie dodatkowego enkodera do mapowania obrazów do przestrzeni latentnej. W f-ANOGAN proces odwrotnej projekcji nie wymaga iteracyjnej optymalizacji, a do wykrywania anomalii używa się łączonego błędu rekonstrukcji i błędu cech z dyskriminatora. Inny pokrewny model, GANomaly (2018), łączy autoenkoder z GAN-em: encoder-decoder-encoder pozwala uczyć się mapowania obrazu na wektor latentny i odwrotnie, a większe odległości w przestrzeni latentnej przy nieznanach (anomalii) przykładach wskazują na odchylenie od normy. Choć GANomaly i f-ANOGAN powstały dla obrazów, pokazują rozwijane podejścia zwiększające efektywność i dokładność detekcji anomalii.

2.4 Połączenia ANOGAN z modelami LLM w wykrywaniu fake newsów

Nowe duże modele językowe (LLM), takie jak BERT czy GPT, stanowią źródło bogatych reprezentacji semantycznych tekstu. Mogą one wspomagać podejścia oparte na ANOGAN na różne sposoby. Po pierwsze, LLM mogą dostarczać wektorów cech (embeddingów) dla zdań – trenowanie ANOGANA w przestrzeni tych wektorów pozwoliłoby wychwycić anomalie semantyczne. Po drugie, LLM same w sobie oceniają wiarygodność tekstu; okazało się, że fałszywe twierdzenia mają wyższy „perplexity” (mniejsze prawdopodobieństwo) według modelu językowego niż prawdziwe informacje. Kang i in. demonstrują, że jeśli LLM jest najpierw „nakarmiony” wiarygodnymi dowodami, to zdebunkowanie fałszu można traktować jako zadanie minimalizacji perplexity: zdania niezgodne z prawdą są trudniejsze do poprawnego „przypasowania” przez model, co objawia się większym błędem.

2.5 Ograniczenia i kierunki przyszłych badań

Podejście ANOGAN do fake newsów ma jednak wiele wyzwań. Po pierwsze, oryginalny ANOGAN przeznaczony jest dla danych ciągłych (obrazy) – generowanie realistycznych sekwencji słów GAN-ami jest trudne z powodu dyskretnej natury tekstu, co utrudnia propagację gradientu. Zastosowanie hybryd (np. RNN jako generator) pomaga, ale nadal brakuje sprawdzonych mechanizmów do trenowania modeli GAN w pełni na tekstach. Po drugie, fenomen fake news często związany jest z subtelnymi faktograficznymi błędami czy manipulacjami kontekstem, które mogą nie manifestować się jako odchylenie w „statystycznej” warstwie języka. Modele oparte wyłącznie na anomaliach semantycznych mogłyby przeoczyć bardzo przekonujące fake newsy napisane w prawidłowym stylu językowym. Po trzecie, ANOGAN wymaga dużego zbioru „czy-

stych"(uczciwych) danych do nauki – w przypadku treści newsowych kosztowne jest ręczne zapewnienie, że zbiór treningowy nie zawiera żadnej dezinformacji.

Przyszłe prace mogą iść kilkoma ścieżkami. Integracja ANOGAN z LLM otwiera nowe możliwości: np. wykorzystanie transformatorów jako generatora tekstu (zamiast konwolucji) oraz jako ekstraktora cech dla dyskriminatora. Połączenie mechanizmów fakt-checkingu (np. weryfikacja zewnętrznych baz wiedzy) z anomalijnymi wskaźnikami językowymi może poprawić trafność detekcji. Ponadto, szybkie wersje ANOGAN (jak f-ANOGAN) redukują czas inferencji dzięki trenowaniu dodatkowego enkodera – podobne przyspieszenia warto zaimplementować w tekstowych adaptacjach. Wreszcie, wskazówką są prace nad detekcją semantycznych anomalii z użyciem LLM: mogą one sugerować, jak mierzyć niespójność logiczną lub faktograficzną w newsach. W sumie, hybrydyczne systemy łączące nienadzorowane uczenie anomalii (GAN) z nadzorowanymi modelami językowymi oraz ekspercką wiedzą wydają się najbardziej obiecującym kierunkiem dalszych badań.

3 implementacja modelu ANOGAN z LSTM

3.1 Implementacja LSTM i próba stworzenia mechanizmu Attention w NumPy

1. Inicjalizacja wag i biasów czterech bramek

W konstruktorze `__init__` stworzyłem macierze W_f , W_i , W_c , W_o o wymiarach $(\text{hidden_size} \times (\text{input_size} + \text{hidden_size}))$ i wektory przesunięć b_f , b_i , b_c , b_o o wymiarach $(\text{hidden_size} \times 1)$.

Dzięki temu każda bramka LSTM (forget, input, candidate, output) ma własny zestaw parametrów, co wiernie odtwarza oryginalny wzór sieci.

2. Metoda forward – pojedynczy krok czasowy

Połączyłem poprzedni stan ukryty h_{prev} i bieżące wejście x przez konkatencję pionową (`np.vstack`), co odpowiada $[h_{t-1}, x_t]$.

Obliczyłem wskaźniki „forget” (f_t), „input” (i_t), kandydat na nową pamięć (c_{tilde}) i „output” (o_t) przy pomocy sigmoidy oraz tanh.

Zaktualizowałem stan komórki $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ i stan ukryty $h_t = o_t \odot \tanh(c_t)$.

Dlaczego to kluczowe? Ręczna implementacja pokazuje, jak poszczególne bramki kontrolują przepływ informacji i utrzymywanie kontekstu w sekwencjach.

3. Metoda forward_sequence – przetwarzanie całej sekwencji

Przeszedłem przez każdy wektor x_t w sekwencji x_{seq} (o kształcie $(\text{seq_len}, \text{input_size})$), wywołując `forward` i zbierając kolejne stany ukryte do listy.

Na wyjściu zwróciłem macierz $(\text{hidden_size} \times \text{seq_len})$ z wszystkimi stanami oraz ostatnie h, c .

Dlaczego? Potrzebuję pełnego przebiegu sekwencji zarówno do wizualizacji wyników, jak i do mechanizmu uwagi.

4. Rozszerzenie: LSTMWithAttention

W konstruktorze dodałem niewielką macierz $\mathbf{W}_a$ ($1 \times \text{hidden_size}$) i bias $\mathbf{b}_a$ (1×1) dla score'u uwagi.

W `forward_with_attention` najpierw wywołałem `forward_sequence`, by uzyskać wszystkie $\mathbf{h}_t$.

Obliczyłem wektor score'ów: $\text{scores} = \tanh(\mathbf{W}_a \cdot \text{outputs} + \mathbf{b}_a)$, a potem softmax wzdłuż wymiaru sekwencji, by uzyskać wagi `att_weights`.

Ostatecznie skróciłem macierz ($\text{hidden_size} \times \text{seq_len}$) do wektora kontekstowego ($\text{hidden_size} \times 1$) ważonym sumowaniem.

Kluczowe: dzięki temu nie polegam tylko na ostatnim stanie LSTM, ale uwzględniam całą historię, ważoną zgodnie z istotnością.

3.2 Tokenizacja i uproszczony BPE

1. `convert_text_to_bytes`

Konwertuję każdy znak na odpowiadający mu bajt UTF-8. Uzyskuję listę liczb całkowitych, które mogę potem traktować jak „tokeny”.

2. `merge_pair_single`

Dla podanej pary (a, b) skanuję listę tokenów i za każdym razem, gdy napotkam sąsiadującą parę równą (a, b) , zastępuję ją nowym tokenem `new_token`.

- Pozwala to łączyć często występujące bigramy w pojedynczy symbol.

3. `iterative_merge`

W pętli określonej przez `num_iterations` liczę częstotliwość każdej pary, wybieram tę najpopularniejszą i scałem ją raz funkcją `merge_pair_single`.

Po każdej iteracji zwiększam wartość identyfikatora nowego tokenu.

Dlaczego to kluczowe? Pozwala zredukować długość sekwencji i zmniejszyć słownik bez konieczności ręcznego definiowania reguł, co poprawia wydajność kolejnych warstw modelu.

3.3 Architektury Generатора i Dyskryminatora w PyTorch

1. Generator (`LSTMGeneratorPyTorch`)

Przyjąłem losowy wektor $\mathbf{z}$ (`latent_dim`) i przekształciłem go przez warstwę `Linear` $\rightarrow$ `reshape` $\rightarrow$ `LSTM` $\rightarrow$ `Linear` do sekwencji embeddingów.

To umożliwia modelowi generowanie sekwencji o długości `seq_len`, na którą składa się zarówno tytuł, jak i treść w postaci wektorów.

2. Dyskryminator (`DiscriminatorPyTorch`)

Wziąłem sekwencję osadzonych wektorów, przepuściłem przez dwukierunkowy LSTM, a następnie uśredniłem (mean pooling) jego wyjścia wzdłuż osi czasowej.

Przez liniową warstwę i sigmoidę przekształcam tę uśrednioną reprezentację do pojedynczej wartości: prawdopodobieństwa bycia „prawdziwą” sekwencją.

Cel: generator i dyskryminator walczą – generator próbuje oszukać dyskryminator, dyskryminator staje się coraz lepszy w wykrywaniu wygenerowanych próbek.

3.4 Wczytanie i podział danych

1. Wczytałem plik CSV i wyciągnąłem kolumny `text`, `title`, `label`.
2. Oddzieliłem wiadomości `REAL` i `FAKE`.
3. Zrealizowałem **dwustopniowy split**: Połowę próbek `REAL` wrzuciłem do `dfReal`. Pozostałych `REAL` połączyłem z całą grupą `FAKE`, a następnie ponownie pociąłem na połowy, tworząc `dfFake`.
4. Dzięki temu oba `DataFrame`’y mają porównywalne rozmiary, redukując ryzyko dominacji jednej klasy w trakcie treningu.

3.5 Zastosowanie tokenizacji i BPE

Dla każdej ramki (`dfReal`, `dfFake`):

1. Zamieniłem kolumny `title` i `text` na listy bajtów.
2. Przeprowadziłem pięć iteracji łączenia par (funkcja `iterative_merge`).

Rezultat: każda wiadomość ma teraz dwie listy tokenów, znacznie krótsze od pierwotnego ciągu bajtów, ale zawierające najczęstsze bigramy.

3.6 Warstwa embeddingów

1. Zebrałem wszystkie unikalne tokeny z obu `DataFrame`’ów, by ustalić `num_embeddings`.
2. Stworzyłem `nn.Embedding(num_embeddings, embedding_dim=5)`.
3. Dla każdej sekwencji tokenów (tytułu i tekstu) wygenerowałem 5-wymiarowe embeddingi i zapisałem je jako `NumPy`.

Dlaczego? Gęste wektorowe reprezentacje poprawiają zdolność LSTM do wychwytywania semantyki danych tekstowych.

3.7 Ekstrakcja stanów ukrytych z LSTM+Attention

Korzystając z `LSTMWithAttention`, dla każdej sekwencji embeddingów:

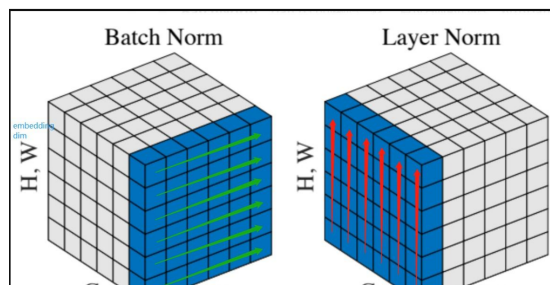
1. Uzyskałem pełną macierz stanów ukrytych ($\text{hidden_size} \times \text{seq_len}$).
2. Obliczyłem wektor uwagi i wyciągnąłem kontekstowy wektor.
3. Zwróciłem też finalny stan ukryty.

Efekt: każdy tekst i każdy tytuł reprezentuję dwoma wektorami ($\text{hidden_size} \times 1$), które skumulowały informację z całej sekwencji.

3.8 Scalanie i normalizacja

1. Ułożyłem pary wektorów $[\text{stan_tytułu}, \text{stan_tekstu}]$ w macierz ($2 \times \text{hidden_size}$).
2. Przekształciłem je na tensory pytorchowskie i zastosowałem `LayerNorm` wzdłuż wymiaru `hidden_size`, by wyrównać rozkład wartości.

Wizualizacja oraz porównanie do Batch normalization:



Normalizacja stabilizuje naukę GAN-a, bo ani generator, ani dyskryminator nie mają „przewagi” wynikającej z różnej skali wejść.

3.9 Przygotowanie DataLoaderów

Z `real_embeddings` oraz `fake_embeddings` utworzyłem `TensorDataset` i `DataLoader` z `batch_size=16` i `shuffle=True`.

Dlaczego? PyTorch `DataLoader` zapewnia wydajne podawanie partii danych z losowym przetasowaniem, co jest krytyczne do stabilnego treningu.

3.10 Inicjalizacja modeli, optymalizatorów i funkcji straty

1. Przeniosłem generator i dyskryminator na GPU (jeśli dostępne).
2. Ustawiłem optymalizatory `Adam(lr=0.0002)` dla obu sieci.
3. Wybrałem `BCELoss` jako funkcję mierzącą różnicę między predykcjami dyskryminatora a etykietami.

Cel: przygotować środowisko do treningu naprzemiennego obu sieci.

3.11 Funkcja `train_step`

1. Dyskryminator

Obliczam stratę na prawdziwych przykładach (`label=1`) i na wygenerowanych bez aktualizacji generatora (`label=0`). Suma strat (średnia) to ostateczny loss_D , którego gradienty aktualizują tylko parametry dyskriminatora.

2. Generator

Zamrażam dyskriminator, generuję nowe próbki z losowego $\mathbf{z}$ i nakładam etykietę 1 (generator „oszukuje”). $\text{loss}_G = \text{BCELoss}(\text{discriminator}(\text{generator}(\mathbf{z})), 1)$, który aktualizuje parametry tylko generatora.

Dlaczego? To klasyczny schemat treningu GAN: dyskriminator i generator uczą się na przemian, co prowadzi do wzajemnej konkurencji i coraz lepszych generowanych próbek.

4 Pętla treningowa

Przez **2100 epok**: dla każdej partii danych z prawdziwych przykładów wywołuję `train_step`, agreguję średnie straty i co 10 epok wypisuję postęp.

Wynik: dwie listy `losses_D` i `losses_G`, które potem wizualizowałem (wcześniej pokazywałeś wykres).

4.1 Detekcja anomalii i wizualizacja rozkładu

1. Procedura detekcji

Ustawiam `discriminator.eval()`.

Przełączam dyskriminator w tryb ewaluacji, żeby wyłączyć Dropout i zatrzymać aktualizację statystyk BatchNorm. W fazie detekcji nie chcę, żeby dyskriminator się uczył ani zmieniał swojego stanu.

Przełączam `generator.train()` przed optymalizacją $\mathbf{z}$.

Choć nie trenuję wag generatora, potrzebuję, aby warstwy BatchNorm w generatorze działały w trybie treningowym, zbierając statystyki z bieżącego batcha. Dzięki temu podczas optymalizacji wektora $\mathbf{z}$ używam poprawnych, „świeżych” statystyk.

Optymalizuję wektor $\mathbf{z}$ (`requires_grad=True`).

Przez ustaloną liczbę kroków, na przykład 100, optymalizuję $\mathbf{z}$, minimalizując MSELoss między `generator( $\mathbf{z}$ )` a moją próbką `sample`. W ten sposób odnajduję takie $\mathbf{z}$, które najlepiej rekonstruuje dane wejście.

Przełączam `generator.eval()` po optymalizacji.

Zanim policzę ostateczne straty, ustawiam generator w trybie ewaluacji, żeby wyłączyć Dropout i użyć stałych statystyk BatchNorm. Dzięki temu ocena rekonstrukcji jest deterministyczna.

Obliczam końcowy „anomaly score”:

$$\text{residual_loss} = \text{MSELoss}(\text{generator}(\mathbf{z}), \text{sample})$$

$$\text{discriminator_score} = D(\text{sample})$$

$$\text{anomaly_score} = (1 - \text{discriminator_score}) + \lambda \cdot \text{residual_loss}$$

Jeśli $D(\text{sample})$ jest bliskie 0, to $1 - D(\text{sample})$ jest duże, co sygnalizuje anomalię. Jeśli generator nie potrafi dobrze odtworzyć sample, residual_loss jest duże, co również wskazuje na anomalię.

2. Wskaźnik anomalii

- residual_loss = MSE(generator($\mathbf{z}$), sample) - discriminator_score = $D(\text{sample})$
- anomaly_score = $(1 - \text{discriminator_score}) + \lambda \cdot \text{residual_loss}$

3. Wizualizacja

Rysuję histogramy rozkładów **real_scores** i **fake_scores**. Ustawiam próg na 80-percentylu prawdziwych i liczę, ile punktów przekracza go w każdej klasie.

Dlaczego kluczowe? ANOGAN wykorzystuje jednocześnie zdolność generatora do rekonstrukcji i dyskryminatora do oceny autentyczności, co daje mocną miarę odchylenia od „normalnego” rozkładu danych.

Optymalizacja przy pomocy "Adam" pozwala modelowi ominąć problem wpadnięcia w minimum lokalne. Adam to wydajny algorytm używany do trenowania modeli uczenia maszynowego, szczególnie w uczeniu głębokim. Łączy w sobie zalety dwóch innych metod, Momentum i RMSprop, aby adaptacyjnie dostosować szybkość uczenia się dla każdego parametru, co prowadzi do szybszej konwergencji i poprawy wydajności.

Minimum lokalne = punkt niemalejący wokół siebie, ale wyższy od globalnego dna.

Minimum globalne = najniższy punkt w całej przestrzeni.

W kontekście trenowania sieci: unikamy lokalnych minimów, by model osiągnął jak najniższą stratę.

Dlaczego to ma znaczenie ?

Podczas trenowania sieci neuronowej optymalizator (np. właśnie Adam) minimalizuje funkcję straty – szuka najniższego punktu. Jeśli wpadnie w minimum lokalne, może „utknąć” i nie znaleźć lepszego globalnego rozwiązania. To oznacza, że model może nie osiągnąć możliwie najlepszej jakości.

5 Problemy, spostrzeżenia, dokładniejsze opisy

1. LSTM z mechanizmem Attention

Problem z czystym LSTM polega na tym, że choć sam potrafi wyciągnąć sekwencyjny kontekst, to przy długich tekstach cała informacja często kumuluje się w ostatnim stanie ukrytym. To sprawia, że wcześniejsze ważne fragmenty mogą zostać „zapomniane” lub przyćmione przez późniejsze elementy sekwencji.

Moim rozwiązaniem jest mechanizm uwagi. Pozwalam modelowi „zwrócić wzrok” na wszystkie kroki sekwencji równomiernie, przyznając im wagi proporcjonalne do ich istotności. Dzięki temu osiągam lepszą reprezentację, ponieważ każda pozycja w tekście ma szansę wpłynąć na ostateczny wektor reprezentujący całą sekwencję. Co więcej, zyskuję wyjaśnialność – mogę sprawdzić, które fragmenty tekstu model uznał za kluczowe, patrząc na rozkład wag attention.

W moim projekcie to właśnie uwaga pozwala mi „wyciągnąć” te części tytułu i treści, które najbardziej różnicują prawdziwe wiadomości od fałszywych. Bez tej warstwy ryzyko utraty sygnału w długich wiadomościach byłoby znacznie większe.

2. Byte-Pair Encoding (BPE)

Problem, który napotykam, dotyczy słów i znaków. Pełny zestaw znaków UTF-8 (lub słów) może być ogromny, co prowadzi do bardzo dużych „słowników” (miliony tokenów). To spowalnia zarówno embedding, jak i kolejne warstwy sieci.

Moim prostym rozwiązaniem jest BPE. Iteracyjnie łączę najczęstsze pary bajtów, co redukuje długość sekwencji i liczbę unikalnych tokenów, pozostawiając w słowniku tylko te n-gramy, które rzeczywiście się powtarzają. Przynosi to korzyści takie jak mniejszy słownik, co oznacza szybsze uczenie warstwy embedding i mniejszą potrzebę pamięci. Jednocześnie zachowuję kluczowe n-gramy – najczęstsze bigramy (np. „th”, „in”, „ed”) stają się pojedynczymi tokenami, co zachowuje sens semantyczny, lecz oszczędza miejsce.

W moim projekcie BPE ułatwia modelowi wychwytywanie wzorców tekstowych charakterystycznych dla fake news (np. często powtarzających się fraz) bez przeciążania sieci zbyt długimi wektorami.

3. Embeddingi

Zauważyłem, że Generator i Dyskryminator działają na wartościach ciągłych. Bezpośrednie użycie tokenów (liczb całkowitych) nie miałoby sensu w warstwach LSTM czy liniowych.

Moim rozwiązaniem jest warstwa Embedding. Każdy token (bajt, bigram) jest mapowany na gęsty wektor n-wymiarowy. Te wektory uczą się odwzorowywać podobieństwa (np. bigramy o zbliżonym znaczeniu znajdują się blisko w przestrzeni) i pozwalają LSTM operować na wartościach rzeczywistych, co jest niezbędne do gradientowego trenowania.

W moim projekcie embeddingi stanowią „wejście wysokiego poziomu” dla LSTM+Attention i dla Generatora/Dyskryminatora w GAN, pozwalając modelom uczyć się semantyki tekstu, a nie tylko sekwencji surowych liczb.

4. Scalanie stanów tytułu i treści oraz normalizacja (LayerNorm)

Każda wiadomość, z którą pracuję, ma dwa źródła informacji: tytuł i treść. Moim podejściem jest wspólne zestawienie ich stanów ukrytych w jedną strukturę, co pozwala GAN-owi ocenić cały kontekst, a nie tylko jeden aspekt danych.

Jednak napotykam problem z różnymi rozkładami wartości z LSTM, które w różnych skalach mogą wprowadzać niestabilność podczas treningu GAN-a. Generatorowi i dyskryminatorowi trudniej się „dogadać”, jeśli ich wejścia krążą wokół różnych średnich i odchyłeń.

Moim rozwiązaniem jest LayerNorm. Wyrównuję średnią i wariancję wektorów wzdłuż wymiaru ukrytego, co stabilizuje gradienty w GAN-ie, umożliwia szybsze zbieganie się modelu i zapobiega „przeskakiwaniu” jednych wejść nad drugimi.

W moim projekcie dzięki znormalizowanym, połączonym wektorom model łatwiej identyfikuje subtelne różnice między prawdziwymi a fałszywymi wiadomościami, a trening jest bardziej przewidywalny.

5. Procedura detekcji anomalii (ANOGAN)

1. **Ustawiam `discriminator.eval()`** Przełączam dyskryminator w tryb ewaluacji, żeby wyłączyć Dropout i zatrzymać aktualizację statystyk BatchNorm – w fazie detekcji nie chcę, żeby dyskryminator się uczył ani zmieniał swojego stanu.
2. **Przełączam `generator.train()` przed optymalizacją** z Choć nie trenuję wag generatora, potrzebuję, aby warstwy BatchNorm w generatorze działały w trybie treningowym (zbierały statystyki z bieżącego batcha). Dzięki temu podczas optymalizacji wektora z używam poprawnych, „świeżych” statystyk.
3. **Optymalizuję wektor z („latent inversion”)**
 - Losuję wektor z z `requires_grad=True`.
 - Przez ustaloną liczbę kroków (np. 100) optymalizuję z , minimalizując **MSELoss** między `generator(z)` a moją próbką `sample`. W ten sposób odnajduję takie z^* , które najlepiej rekonstruuje dane wejście.
4. **Przełączam `generator.eval()` po optymalizacji** Zanim policzę ostateczne straty, ustawiam generator w trybie ewaluacji, żeby wyłączyć Dropout i użyć stałych statystyk BatchNorm – dzięki temu ocena rekonstrukcji jest deterministyczna.

5. Obliczam końcowy „anomaly score”

```
residual_loss = MSELoss(generator(z*), sample)
discriminator_score = D(sample)
anomaly_score = (1 - discriminator_score) +  $\lambda$  · residual_loss
```

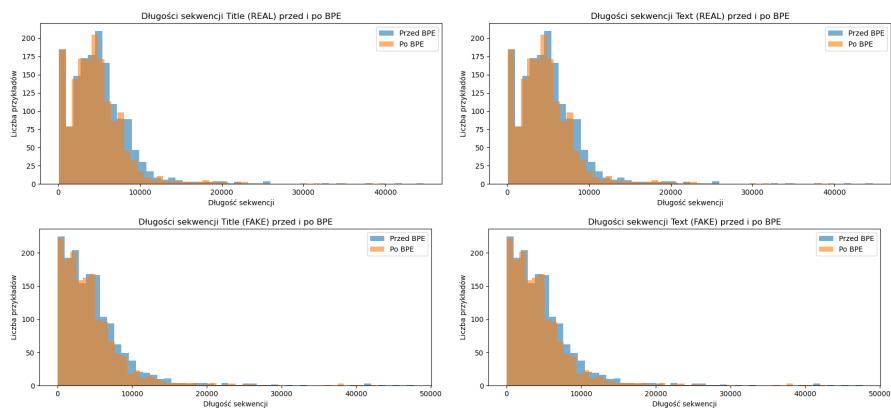
- Jeśli $D(\text{sample})$ jest bliskie 0, to $1 - D(\text{sample})$ jest duże $\rightarrow$ sygnał anomalii.
- Jeśli **generator** nie potrafi dobrze odtworzyć **sample**, **residual_loss** jest duże $\rightarrow$ również wskazuje na anomalię.

Dzięki takiemu podejściu łączę informację o tym, jak bardzo dyskryminator uważa próbkę za „nierealną” z tym, jak słabo generator potrafi ją zrekonstruować.

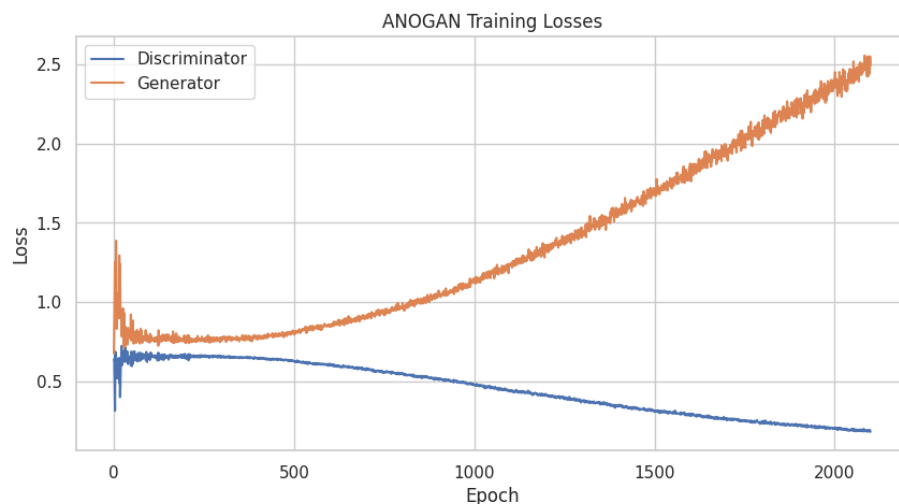
Wizualizacja przebiegu uczenia oraz opis

Algorytm BPE jest dobry, ale nie można przesadzić z "mocą" skracania, ponieważ później nadmierna liczba iteracji prowadzi do tworzenia długich, rzadkich tokenów, które rozmywają semantykę i zwiększają koszt obliczeniowy. Kluczem jest więc znalezienie kompromisu między kompresją a zdolnością modelu do generalizacji.

Wizualizacja algorytmu BPE dla różnych danych ze zbioru:



Histogram pokazuje rozkład długości sekwencji tekstowych oznaczonych jako FAKE przed i po zastosowaniu techniki BPE (Byte Pair Encoding), gdzie oś X przedstawia długość sekwencji, a oś Y liczbę przykładów. Widać, że po BPE (pomarańczowe słupki) sekwencje stają się znacznie krótsze - większość ma długość poniżej 5000 tokenów, podczas gdy przed BPE (niebieskie słupki) rozkład był bardziej rozproszony z sekwencjami sięgającymi nawet 15000 tokenów.



Przebieg uczenia się modeli GAN rywalizujących ze sobą

Ten wykres przedstawia **przebieg trenowania sieci ANOGAN (anomaly detection GAN)**, pokazując, jak zmieniają się **straty (loss)** dwóch komponentów sieci GAN — **generatora** i **dyskryminatora** — w czasie (w funkcji epok). Z biegiem czasu zrozumiałem, że ten model prawdopodobnie się przeuczał oraz podczas treningu modele po 200 epoce przestawały rywalizować ze sobą co świadczyło źle o przebiegu treningu.

Co pokazuje wykres:

- **Oś X:** Liczba **epok** treningowych (czyli ile razy dane zostały przetworzone przez model).
- **Oś Y:** Wartość **straty (loss)** – im niższa, tym lepiej model się dopasował do danych (przy założeniu odpowiedniej równowagi).
- **Linie:**
 - **Niebieska (dyskryminator):** Początkowo wysoka, a potem **stopniowo maleje**.
 - **Pomarańczowa (generator):** Na początku spada, ale później **systematycznie rośnie**.

Interpretacja treningu ANOGAN:

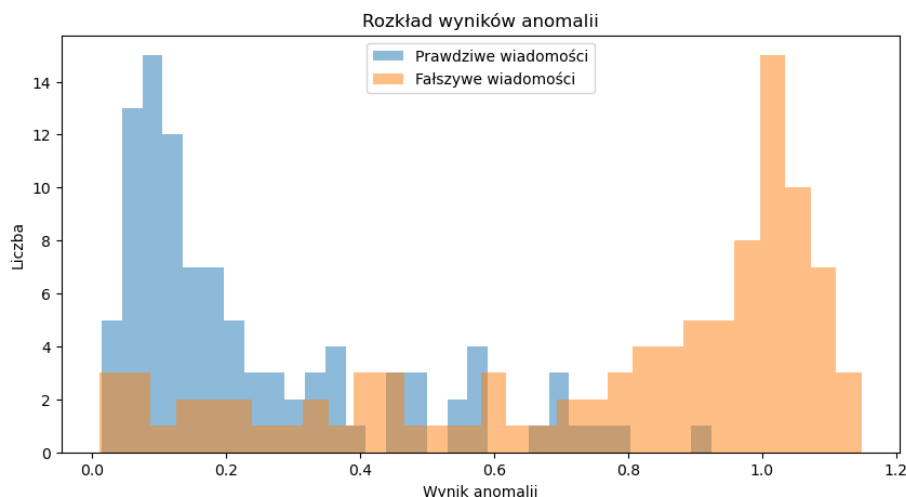
1. Na początku treningu:

- Straty obu sieci są dość wysokie i niestabilne.

- Generator szybko się uczy i poprawia swoje wyniki – jego strata maleje.
- Dyskryminator również się poprawia i dostosowuje do rozpoznawania wygenerowanych próbek.

2. Po pewnym czasie:

- **Dyskryminator coraz lepiej odróżnia dane rzeczywiste od fałszywych** – jego strata spada stabilnie.
- **Generator zaczyna tracić „walkę” z dyskryminatorem**, co powoduje, że jego strata rośnie – generowane dane są coraz łatwiejsze do wykrycia jako fałszywe.



Co oznacza wykres:

- **Oś X:** "Wynik anomalii" – prawdopodobnie wartość od 0 (normalna wiadomość) do 1 (anomalna wiadomość).
- **Oś Y:** Liczba wiadomości (czyli ile wiadomości miało dany wynik).
- **Niebieski słupek:** Prawdziwe wiadomości
- **Pomarańczowy słupek:** Fałszywe wiadomości

Co z niego wynika:

- **Prawdziwe wiadomości mają niskie wyniki anomalii** (głównie 0.0–0.4), co oznacza, że model uznał je za "normalne".
- **Fałszywe wiadomości mają wysokie wyniki anomalii** (głównie 0.7–1.1), co oznacza, że model uznał je za "anomalne".

6 Istnieje lepszy sposób ?

6.1 Ulepszanie modelu

Poprzedni model podczas uczenia był trenowany na 2100 epokach i było widać znaczący wzrost generatora w porównaniu z dyskryminatorem co oznacza, że generowanie nowych treści było zbyt silne a dyskryminator nie był w stanie odpowiednio szybko uczyć się nowych próbek.

w pierwszej epoce modele osiągały następujące wyniki:

$$Epoch10- > Loss_D : 0.5482, Loss_G : 0.9382$$

gdzie Loss D jest stratą dla dyskryminatora, natomiast loss G jest stratą generatora. a wynik końcowy epok wynosi:

$$Epoch2100- > Loss_D : 0.2030, Loss_G : 2.3119$$

Drugi model, który ma wprowadzone mnóstwo poprawek jest o wiele lepszy ponieważ jego początkowy stan trenowania wynosi prawie tyle co końcowy pierwszego modelu.

6.2 Zmiany jakie nastąpiły

Oczywiście po za zmianami w algorytmie BPE, który został usprawniony i lepszym podziałem danych, same hiperparametry zostały ustawione w następujący sposób:

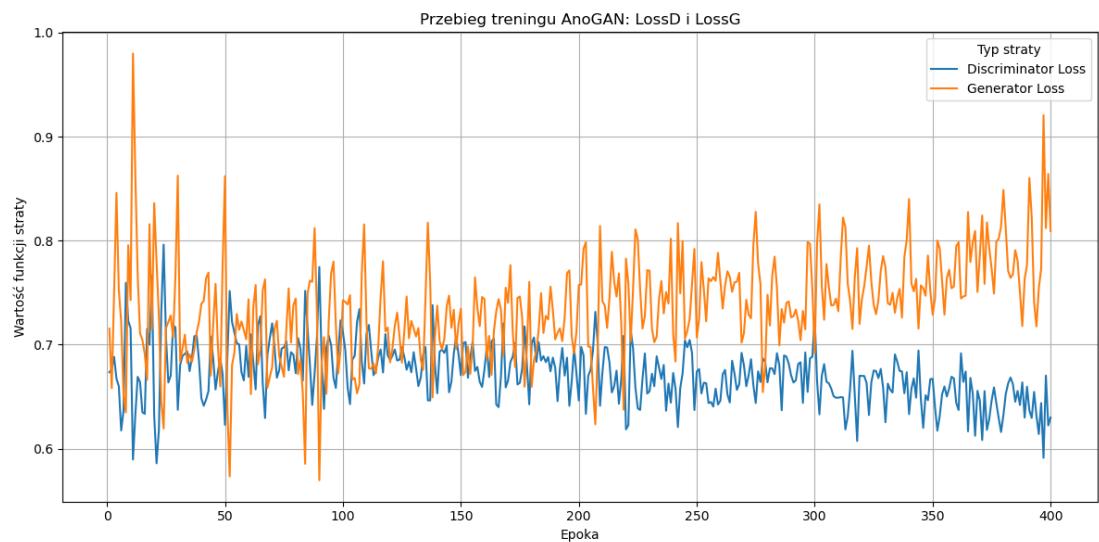
Główne Różnice i Ulepszenia w Modelu 2 1. Architektura Sieci Neuronowych (Generator i Dyskryminator) Najważniejsze zmiany zaszły w budowie obu sieci. Model 2 wykorzystuje bardziej złożone i nowoczesne architektury.

- Głębsze Sieci: Obie sieci w Modelu 2 (Generator i Dyskryminator) są głębsze, co oznacza, że mają więcej warstw ukrytych. Pozwala to modelowi na naukę bardziej skomplikowanych i subtelnych wzorców w danych.
- Normalizacja Wsadowa (Batch Normalization): W Generatorze Modelu 2 dodano warstwy BatchNorm1d. Pomaga to w stabilizacji i przyspieszeniu procesu uczenia, zapobiegając problemowi "zmiennej dystrybucji wewnętrznych aktywacji"(internal covariate shift). W efekcie trening jest bardziej płynny.
- Warstwy Dropout: W Dyskryminatorze Modelu 2 zastosowano warstwy Dropout. Jest to technika regularyzacji, która zapobiega przeuczeniu (overfittingowi) poprzez losowe "wyłączanie" części neuronów podczas treningu. Zmusza to sieć do nauki bardziej odpornych i uogólnionych cech.
- Funkcja Aktywacji LeakyReLU: W obu sieciach Modelu 2 użyto funkcji LeakyReLU zamiast standardowej ReLU. Zapobiega ona problemowi "martwych neuronów"(dying ReLU) poprzez pozwolenie na niewielkie, nieliniowy przepływ gradientu dla ujemnych wartości. To często prowadzi do lepszych wyników w sieciach GAN.

Kluczowe są również zmiany hiperparametrów parametrów:

1. opt steps z 5 na 10
2. embedding dim z 50 na 100
3. batch size z 16 na 32
4. Liczba epok z 2100 na 400

UWAGA bawienie się trsholdem daje bardzo skrajne wyniki np. model świetnie rozpoznaje fake news ale nie rozpoznaje tak dobrze normalnych wiadomości co pokazuje jak bardzo czuły jest to parametr. threshold wynosił 0.5 i dawało to taki efekt:



Opis techniczny wykresu:

Wykres przedstawia wartości funkcji straty (loss) dla dwóch komponentów sieci AnoGAN: Generatora oraz Dyskryminatora, w funkcji liczby epok (do 400). Oś X oznacza kolejne epoki treningowe, natomiast oś Y przedstawia wartości funkcji straty, mieszczące się w zakresie od około 0.55 do 1.0. Dwa kolory reprezentują odpowiednio:

Niebieski: funkcja straty Dyskryminatora (Discriminator Loss)

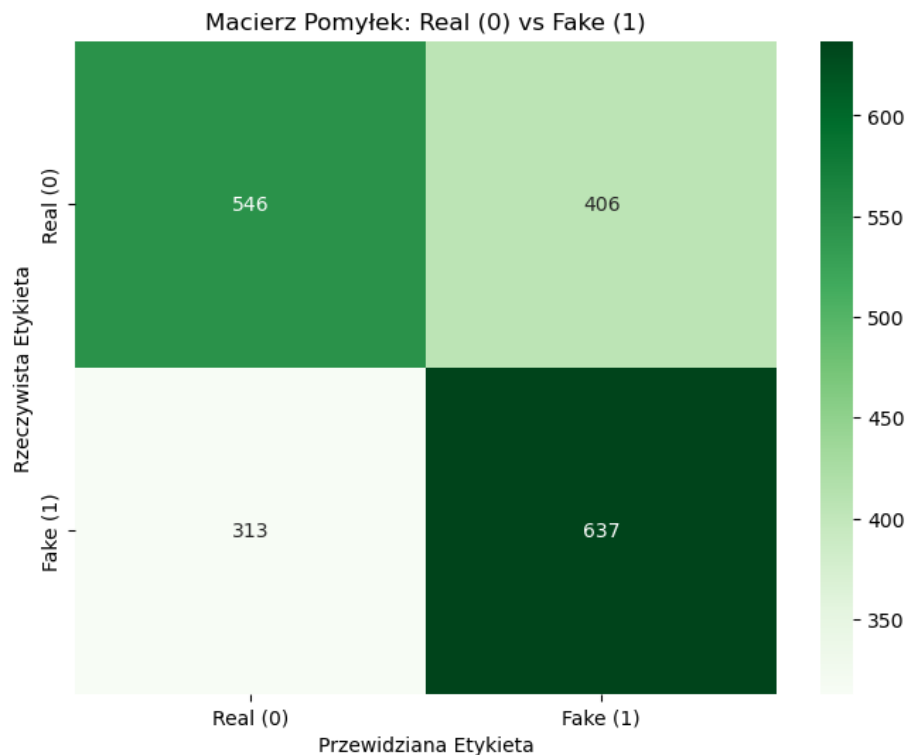
Pomarańczowy: funkcja straty Generatora (Generator Loss)

Analiza i interpretacja:

Stabilność treningu: Obie funkcje straty oscylują wokół wartości średnich (ok. 0.65–0.75), co jest charakterystyczne dla zbalansowanego treningu sieci GAN. Oznacza to, że ani Generator, ani Dyskryminator nie dominują — nie obserwujemy silnego spadku jednej funkcji straty przy jednoczesnym wzroście drugiej, co mogłoby świadczyć o niezrównoważonym treningu.

Generator stopniowo się poprawia: Widać tendencję do lekkiego zwiększenia lossu Generatora z czasem, co przy jednoczesnym delikatnym spadku lossu Dyskryminatora może oznaczać, że model generuje coraz bardziej realistyczne dane, które Dyskryminator musi rozróżniać z większą precyzją.

Brak niestabilności: Brak gwałtownych skoków wartości straty (>1.0 lub <0.4) jest pozytywnym wskaźnikiem — sugeruje, że proces uczenia przebiega stabilnie, bez eksplozji gradientów czy tzw. "mode collapse".



Oś Y pokazuje rzeczywiste etykiety, oś X przewidywane etykiety. Liczby w komórkach reprezentują ilość przypadków w każdej kategorii.

Wyniki klasyfikacji:

- **Dokładność (Accuracy):** 0,6220
- **Raport klasyfikacji:**
 - **Klasa Real (0):**
 - * Precyzja: 0,64
 - * Czułość (Recall): 0,57
 - * F1-score: 0,60
 - * Liczba przykładów (Support): 952

- **Klasa Fake (1):**
 - * Precyzja: 0,61
 - * Czułość (Recall): 0,67
 - * F1-score: 0,64
 - * Liczba przykładów (Support): 950
- **Makrośrednia (macro avg):**
 - * Precyzja: 0,62
 - * Czułość: 0,62
 - * F1-score: 0,62
- **Średnia ważona (weighted avg):**
 - * Precyzja: 0,62
 - * Czułość: 0,62
 - * F1-score: 0,62

Analiza wydajności: Na podstawie uzyskanych metryk można stwierdzić, że model osiąga umiarkowaną skuteczność w rozróżnianiu przykładów rzeczywistych (Real) od wygenerowanych (Fake). Dokładność klasyfikatora wynosi około 62,2 procent, co wskazuje na stabilną, choć nie idealną, wydajność.

Wyniki klasyfikacji pokazują lekką przewagę modelu w identyfikacji przykładów syntetycznych. Klasa „Fake” osiągnęła wyższą czułość (0,67) oraz F1-score (0,64) w porównaniu do klasy „Real” (czułość: 0,57, F1-score: 0,60). Może to sugerować, że model lepiej radzi sobie z wykrywaniem danych wygenerowanych przez generator, co jest pozytywnym sygnałem w kontekście jego zastosowania w systemie detekcji anomalii, takim jak AnoGAN.

Jednocześnie stosunkowo niska precyzja dla klasy „Fake” (0,61) wskazuje na obecność błędnych klasyfikacji przykładów rzeczywistych jako sztuczne (False Positives). Może to świadczyć o coraz wyższej jakości danych generowanych przez model, które stają się trudne do odróżnienia od rzeczywistych.

Warto również zauważyć, że klasy są dobrze zbalansowane (952 przykładów Real, 950 przykładów Fake), co oznacza, że uzyskane metryki nie są wynikiem nie zrównoważonego zbioru danych.

7 Wnioski i spostrzeżenia

Zmiany wprowadzone w Modelu 2 dają:

- **Stabilność Treningu:** Kombinacja normalizacji wsadowej, niższej szybkości uczenia i funkcji aktywacji LeakyReLU prawdopodobnie przełożyła się na znacznie stabilniejszy proces treningowy. Wykresy funkcji strat dla Modelu 2 powinny pokazywać gładze krzywe i lepszą zbieżność w porównaniu do Modelu 1.

- **Zdolność do Uogólniania:** Użycie Dropoutu w Dyskryminatorze zwiększyło jego zdolność do generalizacji, co oznacza, że lepiej radzi sobie on z nowymi, niewidzianymi wcześniej danymi i jest mniej podatny na przeuczenie.
- **Lepsza Jakość Generowanych Danych:** Dzięki głębszej architekturze i lepszemu treningowi, Generator w Modelu 2 jest w stanie tworzyć dane bardziej zbliżone do rzeczywistych, co jest kluczowe dla skutecznego wykrywania anomalii. Podsumowując, Model 2 jest skuteczniejszy, ponieważ jego architektura jest bardziej zaawansowana, a proces treningu został staranniejsz skalibrowany. Wprowadzone ulepszenia bezpośrednio adresują typowe problemy występujące podczas uczenia sieci GAN, co prowadzi do stabilniejszego procesu i lepszych wyników końcowych.

Model 1 był dobrym punktem wyjścia – stanowił bazową, działającą implementację. Jednak nie spełniał moich oczekiwań dlatego pchnęło mnie to do stworzenia drugiej jak widać lepszej wersji modelu.

Wyniki badań otwierają nowe możliwości dla nienadzorowanych metod wykrywania dezinformacji, które nie wymagają dużych, ręcznie anotowanych zbiorów danych. Takie podejście jest szczególnie wartościowe w kontekście szybko ewoluujących form dezinformacji, gdzie tradycyjne metody nadzorowane mogą okazać się niewystarczające. Ponadto, dyskretna natura tekstu pozostaje fundamentalnym wyzwaniem dla propagacji gradientu w architekturach GAN, co wymaga dalszych badań nad hybrydowymi podejściami łączącymi komponenty symboliczne i subsymboliczne.

Szczególnie istotne jest to, że metodologia ta może być rozszerzona na inne zadania detekcji anomalii tekstowych, takie jak wykrywanie mowy nienawiści, cyberprzemocy czy innych form treści szkodliwych, czyniąc z niej uniwersalne narzędzie analityczne o szerokim zastosowaniu.